

Análisis de Capas de Red

Modelos OSI y TCP/IP — Sistema IoT Bidireccional

Flask (PC) · RPi Zero 2W · ESP32-CAM · Telegram Bot API

1. Descripción del sistema

Topología: LAN 802.11n (192.168.0.0/24) con tres nodos activos: PC Flask dispatcher (192.168.0.8:5000), RPi Zero 2W con Flask+CSI (192.168.0.12:5001) y ESP32-CAM con webserver (192.168.0.10:80). Gateway NAT hacia Internet para acceso a Telegram Bot API (api.telegram.org:443).

Flujo 1 — Sensor → Usuario: Trigger PIR → captura JPEG (CSI/OV2640) → HTTP POST /image (token header) → Flask·PC → HTTPS POST /sendPhoto → Telegram → Usuario.

Flujo 2 — Usuario → Sensor: Usuario envía comando → Telegram → Flask·PC (long polling GET /getUpdates) → routing por comando → HTTP GET /capture → sensor → captura → reinicia Flujo 1.

2. Tabla de capas — Modelo OSI

Cada fila describe la perspectiva de red (protocolos, interfaces, mecanismos) y la función concreta en este sistema. Sin definiciones genéricas.

Capa OSI	Perspectiva de Red	Función específica en el sistema
7 — Aplicación	Protocolos: HTTP/1.1 (LAN), HTTPS/TLS 1.3 (Internet), Telegram Bot API REST (JSON). Puertos activos: TCP/5000 (Flask PC), TCP/5001 (Flask RPi), TCP/80 (ESP32 webserver), TCP/443 (api.telegram.org). Métodos HTTP: POST	Flujo 1 (PIR → Usuario): El nodo sensor (RPi/ESP32) construye un HTTP POST multipart/form-data hacia http://192.168.0.8:5000/image con payload JPEG y token en header. Flask (PC) actúa como proxy de aplicación: valida token, extrae la imagen y emite un HTTP POST HTTPS a api.telegram.org/bot<token>/sendPhoto con el binario JPEG encapsulado. Flujo 2 (Usuario → Sensor): Flask mantiene un bucle de long polling: GET https://api.telegram.org/bot<token>/getUpdates?timeout=N. Al recibir JSON con comando

Capa OSI	Perspectiva de Red	Función específica en el sistema
	/image (Flujo 1), POST /sendPhoto (Telegram), GET /getUpdates (polling), GET /capture (Flujo 2). Autenticación: Token propietario en header HTTP Authorization/X-Token (cleartext sobre LAN).	(/esp32_capture → GET http://192.168.0.10/capture; /rpi_capture → GET http://192.168.0.12:5001/capture), ejecuta routing por valor de texto del campo 'text'. La respuesta del sensor reinicia Flujo 1. Observación: Flask es el único punto de acoplamiento entre la LAN IoT y Telegram. Sin él, ninguno de los dos flujos es posible. El long polling impone una latencia mínima = RTT(PC ↔ api.telegram.org) por ciclo de polling, típicamente 80–300 ms desde Perú.
6 — Presentación	Formatos de datos: JPEG (imagen comprimida con pérdida), JSON (mensajes de control Telegram), multipart/form-data (encapsulación HTTP). Cifrado/codificación: TLS 1.3 en tráfico PC ↔ api.telegram.org. Tramo LAN (sensor ↔ PC) sin cifrado. Serialización: JSON nativo en Telegram API (UTF-8). JPEG generado por libcamera/OpenCV (RPi) u OV2640 firmware (ESP32).	Flujo 1: La cámara CSI del RPi Zero 2W captura un frame y lo comprime en JPEG (calidad y resolución según configuración de libcamera/picamera2). El ESP32-CAM usa el codec interno del OV2640 vía firmware. El binario JPEG se incluye directamente en el body HTTP POST; no hay transcodificación adicional en Flask. Al reenviar a Telegram, Flask entrega el mismo binario JPEG al endpoint /sendPhoto. Flujo 2: La respuesta de /getUpdates es JSON plano UTF-8. Flask parsea el campo message.text para extraer el comando. No hay serialización binaria en este flujo. Observación: El tramo LAN no usa TLS. El token de autenticación viaja en texto plano dentro de la red local. Si el AP está comprometido, el token es recuperable por inspección de tráfico. El tamaño del JPEG (típicamente 20–200 KB según resolución) determina el tiempo de transferencia en la LAN.
5 — Sesión	Gestión de sesiones: HTTP/1.1 Keep-Alive (opcional), TLS session resumption (tramo externo), long polling como sesión semi-persistente. Estado de sesión: Stateless en LAN (cada POST /image es independiente). Pseudo-stateful en polling (conexión TCP larga hacia Telegram). Timeout: Parámetro timeout= en /getUpdates define duración máxima de cada sesión de polling.	Flujo 1: No existe sesión persistente entre capturas. Cada trigger PIR genera un ciclo request/response independiente. El sensor abre una conexión TCP, envía el POST, recibe 200 OK y cierra. Flask hace lo propio hacia Telegram con una nueva conexión (o Keep-Alive si la librería requests lo gestiona). Flujo 2: El long polling crea una sesión TCP de larga duración entre Flask y api.telegram.org. Esta sesión puede durar decenas de segundos o minutos. El riesgo es que el NAT del router descarte el estado de conexión si supera el timeout de NAT (típicamente 30–120 s en routers domésticos), generando un corte silencioso sin notificación a Flask. Observación: Sin mecanismo de heartbeat o reconexión automática ante caída silenciosa del polling, Flask puede quedar en estado 'creyendo que escucha' sin recibir comandos. Esto requiere watchdog o reinicio periódico del hilo de polling.
4 — Transporte	Protocolo único: TCP (RFC 793). No se usa UDP ni QUIC en ningún segmento. Puertos destino: 5000 (Flask PC-Flujo1-RX), 5001 (Flask RPi-Flujo2-TX), 80 (ESP32-Flujo2-TX), 443 (Telegram-ambos flujos). Stack TCP: Linux kernel TCP (PC, RPi Zero 2W). lwIP 2.x (ESP32-CAM, implementación embebida). Control de flujo: Ventana TCP estándar en Linux (~87 KB inicial, escalable). lwIP: ventana por defecto ~5–10 KB por socket.	Flujo 1: El sensor establece un three-way handshake TCP hacia 192.168.0.8:5000. Transfiere el JPEG como stream de bytes sobre TCP. Flask abre un segundo handshake hacia api.telegram.org:443. Total: 2 handshakes TCP + 1 TLS handshake por captura. Latencia acumulada mínima = RTT_LAN×2 + RTT_Internet×3 (TCP+TLS). Flujo 2: La conexión TCP de polling hacia Telegram permanece abierta. Al ejecutar /capture, Flask abre una nueva conexión TCP hacia el dispositivo destino (RPi:5001 o ESP32:80) y espera la respuesta síncrona antes de continuar. Observación crítica (ESP32): lwIP en ESP32-CAM tiene buffers de socket configurados típicamente en 5–10 KB. Una imagen JPEG de 150 KB requiere múltiples segmentos TCP con retransmisiones potenciales si el buffer se satura. Esto puede incrementar la latencia de transferencia en 200–800 ms adicionales respecto al RPi con stack Linux completo.
3 — Red	Protocolo: IPv4 (RFC 791). Sin IPv6 activo (inferido por uso exclusivo de direcciones 192.168.x.x). Topología: Segmento plano 192.168.0.0/24. Sin subredes internas ni VLANs. Direccionamiento: PC: 192.168.0.8 · RPi: 192.168.0.12 · ESP32: 192.168.0.10 · Gateway/NAT: 192.168.0.1	Flujo 1: Los paquetes IP viajan en la LAN con TTL estándar (64 en Linux, 128 en lwIP). El router realiza PAT para reescribir IP/puerto origen al reenviar a Telegram. No hay enrutamiento inter-subred interno. Flujo 2: Mismo gateway NAT. Flask (PC) inicia todas las conexiones salientes, por lo que no se requiere configuración de port forwarding en el router. El enrutamiento interno (PC→RPi, PC→ESP32) se resuelve a nivel L2 (ARP), sin pasar por el gateway. Observación: El gateway es punto único de falla para el flujo externo. En su ausencia, el Flujo 1 (LAN interna) continúa funcionando en sus

Capa OSI	Perspectiva de Red	Función específica en el sistema
	(inferido). NAT: PAT en gateway para tráfico saliente hacia api.telegram.org. Sin port forwarding entrante (el sistema inicia siempre desde LAN).	<i>segmentos locales, pero Telegram queda inaccesible. Sin OSPF, BGP ni redundancia de gateway. Sin QoS/DSCP: el JPEG compite con tráfico genérico por el mismo uplink.</i>
2 — Enlace de datos	Estándar: IEEE 802.11n (Wi-Fi 4), 2.4 GHz. CSMA/CA como mecanismo de acceso al medio. Velocidad de enlace: Hasta 150 Mbps (1 flujo espacial, 40 MHz); throughput real ~50–100 Mbps en condiciones típicas. Frames: Ethernet II encapsulado en 802.11 data frames. MTU 1500 bytes (Ethernet estándar). Direccionamiento L2: MAC: RPi BCM43438, ESP32 (módulo Wi-Fi interno), NIC PC. ARP resuelve IP↔MAC en la LAN.	Flujo 1 y 2: Todos los nodos (RPi, ESP32, PC) son clientes WLAN del mismo AP. El AP actúa como bridge L2: las tramas entre nodos LAN no salen al uplink. El retransmisión CSMA/CA en colisión añade latencia variable (backoff exponencial). Fragmentación: Los frames 802.11n pueden fragmentarse si superan el umbral de fragmentación del AP (por defecto 2346 bytes en muchos routers). Un JPEG de 100 KB se segmenta en ~67 paquetes IP de 1500 bytes, cada uno generando uno o más frames 802.11. Observación: RPi Zero 2W usa antena PCB integrada (BCM43438): ganancia baja (~2 dBi), sensible a obstrucción física. ESP32-CAM tiene antena en chip aún más limitada; algunos modelos incluyen conector u.FL para antena externa. La coubicación de múltiples clientes Wi-Fi en el mismo canal 2.4 GHz incrementa las colisiones CSMA/CA y degrada el throughput efectivo.
1 — Física	Medio inalámbrico: RF 2.4 GHz, OFDM (802.11n PHY). Canales 1, 6 u 11 (no solapados). Interfaces físicas: Antena PCB RPi (BCM43438), antena integrada ESP32, NIC Wi-Fi PC, AP/Router. Bus de cámara: MIPI CSI-2 (cable ribbon FPC 15-pin, 22-pin) entre módulo cámara y RPi Zero 2W. Sensor PIR: Salida digital GPIO (3.3 V TTL). Sin protocolo de red; señal directa al MCU/SBC. Energía: RPi Zero 2W: USB-C 5V/2A. ESP32-CAM: 5V/500 mA mínimo (pico al transmitir Wi-Fi ~300–400 mA).	Flujo 1: El PIR genera un flanco GPIO que interrumpe el proceso Python en el RPi (o la ISR en el ESP32). La cámara CSI (RPi) transfiere datos de imagen a través del bus MIPI CSI-2 hacia la RAM del SoC; el OV2640 (ESP32) entrega el JPEG comprimido via bus DVP interno al SoC. La transmisión Wi-Fi modula los bits en RF a 2.4 GHz con OFDM (hasta MCS7 en 802.11n). Flujo 2: No involucra interfaces físicas locales distintas a la NIC Wi-Fi. La señal de RF del AP demodula los frames de respuesta de Telegram y los entrega al stack de red. Observación: El cable ribbon CSI es el único enlace físico no inalámbrico del sistema y el punto de falla de mayor impacto: su rotura inhabilita toda captura en el RPi. El PIR no tiene confirmación de recepción: si el MCU está ocupado (por ejemplo, en transferencia Wi-Fi), el pulso PIR puede no ser procesado. La inestabilidad de alimentación del ESP32-CAM durante picos de transmisión Wi-Fi es causa frecuente de resets.

3. Tabla de capas — Modelo TCP/IP

El modelo TCP/IP agrupa OSI L1-L2 en 'Acceso a Red' y OSI L5-L7 en 'Aplicación'. Misma precisión de análisis, menor granularidad de capas.

Capa TCP/IP	Perspectiva de Red	Función específica en el sistema
Aplicación(≡ OSI L5+L6+L7)	Protocolos: HTTP/1.1, HTTPS (TLS 1.3), Telegram Bot API REST, JSON, JPEG, multipart/form-data. Endpoints activos:	Esta capa condensa todo el comportamiento de protocolo de usuario: Flujo 1: Sensor → HTTP POST (multipart JPEG + token) → Flask·PC → HTTPS POST /sendPhoto → Telegram → Usuario. Flask actúa como gateway de protocolo: traduce HTTP plano (LAN) a HTTPS (Internet) sin modificar el

Capa TCP/IP	Perspectiva de Red	Función específica en el sistema
	POST /image::5000 · POST /sendPhoto::443 · GET /getUpdates::443 · GET /capture::5001/:80. Autenticación: Token HTTP en header (LAN, cleartext) + token Bot de Telegram (HTTPS).	payload JPEG. Flujo 2: Usuario → comando Telegram → HTTPS GET /getUpdates (long polling, Flask·PC) → parsing JSON → routing por comando → HTTP GET /capture (LAN) → sensor → captura → reinicia Flujo 1. Punto de acoplamiento crítico: Flask es el único componente de aplicación que conoce ambos dominios (LAN IoT + Telegram API). Su caída detiene ambos flujos simultáneamente. El long polling introduce latencia de reacción variable según carga de api.telegram.org y RTT del uplink.
Transporte(= OSI L4)	Protocolo único: TCP (RFC 793). Sin UDP, QUIC ni DTLS en ningún segmento. Conexiones activas simultáneas (estado nominal): 1 TCP long-poll (Flask→Telegram, persistente) + 1 TCP por captura (Flask→sensor, efímera). Stacks: Linux kernel TCP (PC, RPi) · lwIP 2.x (ESP32-CAM). Control de flujo: Ventana TCP Linux: ~87 KB inicial, auto-escalable. lwIP: ~5–10 KB, no escalable por defecto.	Flujo 1 (por captura): SYN→SYN-ACK→ACK (sensor→PC:5000) + transferencia JPEG + FIN. Luego SYN→SYN-ACK→ACK (PC→Telegram:443) + TLS 1.3 handshake (1-RTT si session ticket disponible) + POST /sendPhoto + FIN. Latencia mínima total: 2×RTT_LAN + 3×RTT_WAN. Flujo 2: Conexión TCP persistente PC→Telegram:443 para polling. Al recibir comando, nueva conexión TCP efímera PC→sensor:puerto. Respuesta síncrona bloqueante: Flask espera el 200 OK del /capture antes de procesar el siguiente update de Telegram. Cuello de botella ESP32 (lwIP): Una imagen de 150 KB sobre una ventana de 5 KB requiere ≥30 ciclos ACK. Con RTT_LAN de 1–2 ms, esto añade 30–60 ms solo por el mecanismo de ventana, más el overhead de segmentación TCP. Configurable vía menuconfig (ESP-IDF) aumentando TCP_WND, pero implica mayor uso de DRAM del SoC.
Internet(= OSI L3)	Protocolo: IPv4 (RFC 791). Sin IPv6. Espacio de direcciones: LAN: 192.168.0.0/24 (RFC 1918). Internet: IP pública dinámica del ISP (NAT/PAT en gateway). Enrutamiento interno: Directo en mismo segmento (sin gateway para tráfico LAN↔LAN). Enrutamiento externo: Default route (0.0.0.0/0) → gateway → ISP → api.telegram.org.	Flujo 1 y 2 (segmento LAN): PC (192.168.0.8) ↔ RPi (192.168.0.12) y PC ↔ ESP32 (192.168.0.10) se comunican directamente en L3 sin pasar por el gateway. La resolución de next-hop es ARP directo. TTL de paquetes: 64 (Linux) / 128 (lwIP). Flujo 1 y 2 (segmento Internet): El gateway realiza NAPT: reescribe IP origen (192.168.0.8) → IP pública del ISP y asigna puerto efímero. La respuesta de Telegram sigue la tabla NAT para reenviar al PC. Sin IP pública fija ni DDNS mencionado: el bot siempre inicia conexiones salientes, eludiendo la necesidad de inbound NAT. Observación: La topología flat (sin VLANs) implica que todos los dispositivos IoT (RPi, ESP32) y el PC de control comparten el mismo dominio de broadcast. Un dispositivo comprometido en la LAN tiene acceso directo a todos los demás sin atravesar ningún firewall interno.
Acceso a Red(= OSI L1+L2)	Estándar: IEEE 802.11n (Wi-Fi 4), 2.4 GHz, OFDM. CSMA/CA. Interfaces: BCM43438 (RPi Zero 2W) · módulo Wi-Fi integrado ESP32 (Xtensa LX6 + radio 2.4 GHz) · NIC Wi-Fi PC · AP/Router (bridge L2). MTU: 1500 bytes (Ethernet). Fragmentación IP si payload > MTU. Medio físico adicional: MIPI CSI-2 (ribbon FPC) entre cámara y RPi. GPIO para señal PIR.	Flujo 1 y 2 (Wi-Fi): El AP actúa como bridge 802.11↔Ethernet (si tiene puerto WAN Ethernet) o como gateway Wi-Fi. Todas las estaciones (STA: RPi, ESP32, PC) se asocian al mismo BSSID. El tráfico inter-STA pasa por el AP (no hay comunicación directa STA↔STA en modo infraestructura estándar): cada frame LAN recorre dos saltos RF (STA→AP→STA). Flujo 1 (captura CSI): El módulo cámara transfiere datos por MIPI CSI-2 al SoC del RPi. Este bus opera de forma independiente a la red; su velocidad (típicamente 480 Mbps en CSI-2 de 2 carriles) no es el cuello de botella. La latencia de captura la impone el tiempo de exposición y el procesamiento de compresión JPEG en CPU. Observación: Con RPi Zero 2W + ESP32-CAM + PC en el mismo canal 2.4 GHz, el medio compartido genera contención CSMA/CA. Si ambos sensores intentan transmitir simultáneamente (dos triggers PIR concurrentes), el AP serializa el acceso, incrementando la latencia de uno de los flujos. Sin 5 GHz ni WMM/QoS configurado, no hay priorización posible.

4. Desglose y análisis detallado por capa

Análisis técnico profundo de cada capa enfocado en comportamiento real del sistema, implicancias de diseño, riesgos operativos y cuellos de botella.

Capa 7 / Aplicación TCP-IP — Lógica de aplicación, protocolo de mensajería y autenticación

- ▶ **Flask como dispatcher central:** Flask (Python/Werkzeug) expone el endpoint POST /image y actúa simultáneamente como cliente HTTP hacia los sensores y como cliente HTTPS hacia Telegram. No hay broker de mensajería (MQTT, AMQP): Telegram API cumple esa función de forma implícita mediante long polling.
- ▶ **Autenticación dual:** Token LAN (header HTTP, sin cifrar) para validar sensores; Bot token Telegram (en URL HTTPS, cifrado por TLS) para autenticar ante api.telegram.org. El token LAN es un mecanismo de autenticación básico susceptible a captura por cualquier host en la misma LAN.
- ▶ **Routing por comando:** Flask parsea el campo message.text del JSON de Telegram. La lógica de routing (/esp32_capture → 192.168.0.10, /rpi_capture → 192.168.0.12) está hardcodeada en la aplicación, sin tabla de routing dinámica ni descubrimiento de dispositivos.
- ▶ **Dependencia de disponibilidad externa:** Telegram Bot API es un servicio de terceros con SLA no garantizado. Cualquier degradación de api.telegram.org impacta directamente en ambos flujos. Sin fallback local (p. ej., MQTT broker en LAN).

Capa 6 — Presentación: codificación y cifrado de datos

- ▶ **JPEG como formato de transporte de imagen:** El payload de imagen no se transcodifica en ningún punto. El sensor genera JPEG y Telegram recibe el mismo binario. La calidad y resolución (factor de calidad QV2640 o parámetros libcamera) impactan directamente en el tamaño del payload y en la latencia de transmisión.
- ▶ **TLS 1.3 exclusivamente en el tramo externo:** El tramo LAN (sensor ↔ Flask) es HTTP cleartext. Implicación: el payload JPEG y el token de autenticación son observables por cualquier host en la LAN 192.168.0.0/24 con capacidad de sniffing pasivo (modo monitor Wi-Fi).
- ▶ **JSON como protocolo de control:** Los mensajes de Telegram (comandos, metadatos) viajan como JSON UTF-8. Flask no valida el schema JSON más allá de acceder a campos específicos (message.text), lo que puede generar excepciones no controladas ante respuestas malformadas de la API.

Capa 5 — Sesión: gestión de conexiones y estado

- ▶ **Stateless en Flujo 1:** Cada captura PIR genera un ciclo request/response HTTP completamente independiente. No hay sesión entre capturas. Si Flask está ocupado procesando una captura anterior, el nuevo POST del sensor puede recibir un timeout.
- ▶ **Long polling como sesión semi-persistente:** El GET /getUpdates mantiene una conexión TCP abierta hasta que Telegram devuelve actualizaciones o expira el parámetro timeout=. Flask implementa el polling en un hilo separado (o mediante un bucle bloqueante), lo que puede generar condiciones de carrera si el diseño no es thread-safe.
- ▶ **Riesgo de sesión silenciosa caída:** Los routers domésticos descartan entradas NAT inactivas típicamente entre 30–180 s. Una sesión de polling con timeout=60 puede ser descartada por el NAT antes de que Telegram responda, generando que Flask espere indefinidamente en un socket ya cerrado por el router sin recibir RST.

Capa 4 / Transporte TCP-IP — Control de flujo y entrega confiable

- ▶ **TCP como único protocolo de transporte:** La elección de TCP garantiza entrega ordenada y sin pérdida, adecuada para JPEG (la pérdida de un solo segmento corrompe la imagen). Sin embargo, excluye optimizaciones de baja latencia posibles con UDP+FEC para streaming de video en tiempo real.

► **Handshakes TCP acumulados:** Flujo 1 requiere mínimo 2 handshakes TCP (sensor→Flask, Flask→Telegram) + 1 handshake TLS. En latencias LAN de 1–2 ms y WAN de 80–200 ms, la latencia total solo por establecimiento de conexión es ~85–210 ms por captura. Con TLS session tickets, el handshake TLS se reduce a 1-RTT (~80–200 ms).

► **lwIP en ESP32-CAM:** La implementación lwIP reduce el overhead de memoria en el ESP32 (DRAM total ~520 KB, de los cuales ~200–300 KB son accesibles para heap). El TCP_WND por defecto en lwIP es configurable; en configuraciones ESP-IDF conservadoras puede estar en 5744 bytes. Para transferir una imagen de 100 KB, lwIP necesita ~18 ciclos de ventana completa, cada uno requiriendo un ACK del receptor.

Capa 3 / Internet — Enrutamiento y direccionamiento IP

► **Topología plana sin segmentación:** Todo el tráfico LAN (IoT + control) reside en 192.168.0.0/24. Sin VLANs ni ACLs internas, un dispositivo IoT comprometido puede comunicarse directamente con el PC de control (Flask) sin restricciones de red.

► **NAT como única frontera de seguridad:** El gateway NAT impide conexiones entrantes no solicitadas desde Internet. Sin embargo, el sistema inicia siempre las conexiones (salientes), por lo que el NAT no aporta seguridad adicional contra vectores de ataque desde la LAN.

► **Dependencia de DNS externo:** La resolución de api.telegram.org requiere DNS funcional. Si el DNS del ISP falla o el gateway no tiene resolución en caché, el Flujo 2 se interrumpe incluso si la conectividad IP está disponible. No se menciona DNS local ni fallback.

Capa 2 — Enlace de datos: acceso al medio y entrega de tramas

► **802.11n en modo infraestructura:** Todos los nodos son STAs del mismo AP. El tráfico entre STAs (p. ej., RPi → PC) requiere dos saltos RF: RPi→AP y AP→PC. Esto duplica el uso del medio inalámbrico por cada trama LAN y reduce el throughput efectivo entre STAs a aproximadamente la mitad del throughput teórico del AP.

► **Contención CSMA/CA:** Con tres clientes activos (RPi, ESP32, PC), el acceso al canal se negocia mediante CSMA/CA. Las colisiones generan backoff exponencial binario (BEB), añadiendo latencia variable no determinista. En entornos con alta densidad Wi-Fi (vecinos en el mismo canal), el throughput puede degradarse significativamente.

► **MTU y fragmentación:** El MTU Ethernet de 1500 bytes implica que un JPEG de 100 KB se fragmenta en ~67 paquetes IP, cada uno encapsulado en un frame 802.11. El AP puede agregar frames (A-MPDU en 802.11n) para mejorar eficiencia, dependiendo de la implementación del firmware del AP.

Capa 1 / Acceso a Red físico — Medio de transmisión e interfaces hardware

► **RF 2.4 GHz como único medio de red:** La banda 2.4 GHz es la más saturada en entornos residenciales. Solo 3 canales no solapados (1, 6, 11). Sin 5 GHz disponible en RPi Zero 2W (BCM43438 es solo 2.4 GHz) ni en ESP32-CAM estándar. El sistema está inherentemente limitado a esta banda para todos sus nodos IoT.

► **MIPI CSI-2 (ribbon FPC):** El bus CSI-2 opera a velocidades de hasta 480 Mbps (2 carriles) o 1 Gbps (4 carriles) en la especificación. En la práctica, la latencia de captura en RPi Zero 2W con picamera2 es dominada por el tiempo de compresión JPEG en el CPU ARM Cortex-A53 (single-core efectivo en el Zero 2W para esta tarea).

► **GPIO PIR sin protocolo de red:** La señal PIR es una interrupción hardware. No hay acuse de recibo, no hay confirmación de que el trigger fue procesado. Si el CPU del RPi está en un estado de alta carga (por ejemplo, transmitiendo una imagen anterior), el handler de la interrupción PIR puede encolarse o perderse dependiendo de la prioridad configurada en el sistema operativo.

► **Alimentación ESP32-CAM:** El módulo ESP32-CAM (AI-Thinker) consume hasta 310 mA en transmisión Wi-Fi activa. Fuentes USB de baja calidad (<500 mA) pueden provocar caídas de voltaje durante la transmisión, generando resets del SoC. Esto es causa documentada de inestabilidad en implementaciones de producción.

5. Bibliografía técnica

Postel, J. (1981). *Transmission Control Protocol*. RFC 793. IETF. <https://www.rfc-editor.org/rfc/rfc793>

Postel, J. (1981). *Internet Protocol*. RFC 791. IETF. <https://www.rfc-editor.org/rfc/rfc791>

IEEE Std 802.11n-2009. *Enhancements for Higher Throughput*. IEEE Standards Association.

Dunkels, A. (2001). *Design and Implementation of the lwIP TCP/IP Stack*. Swedish Institute of Computer Science.

Telegram (2024). *Bot API Reference*. <https://core.telegram.org/bots/api>

Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks* (5.^a ed.). Prentice Hall.

Stevens, W. R. (1994). *TCP/IP Illustrated, Vol. 1: The Protocols*. Addison-Wesley.